Rapport de Synthèse — Application de Messagerie Instantanée

Module : ADDA75 — API et Web Services
Formation : Master Ingénierie Informatique
Auteurs :
- Loqmann Benhaddya (Binôme B : Messages, Persistance & Temps Réel WebSocket)
- Binôme (Binôme A : Identité, Sécurité JWT & API REST)

1. Objectif et périmètre

Le projet consiste à faire dialoguer en temps réel deux clients logiciels distincts — un client « bureau » (ordinateur) et un client « mobile » (téléphone) — par l'intermédiaire d'un serveur central que nous avons construit. Les deux appareils ne communiquent jamais directement : le serveur reçoit chaque message, le persiste, puis le retransmet au destinataire.

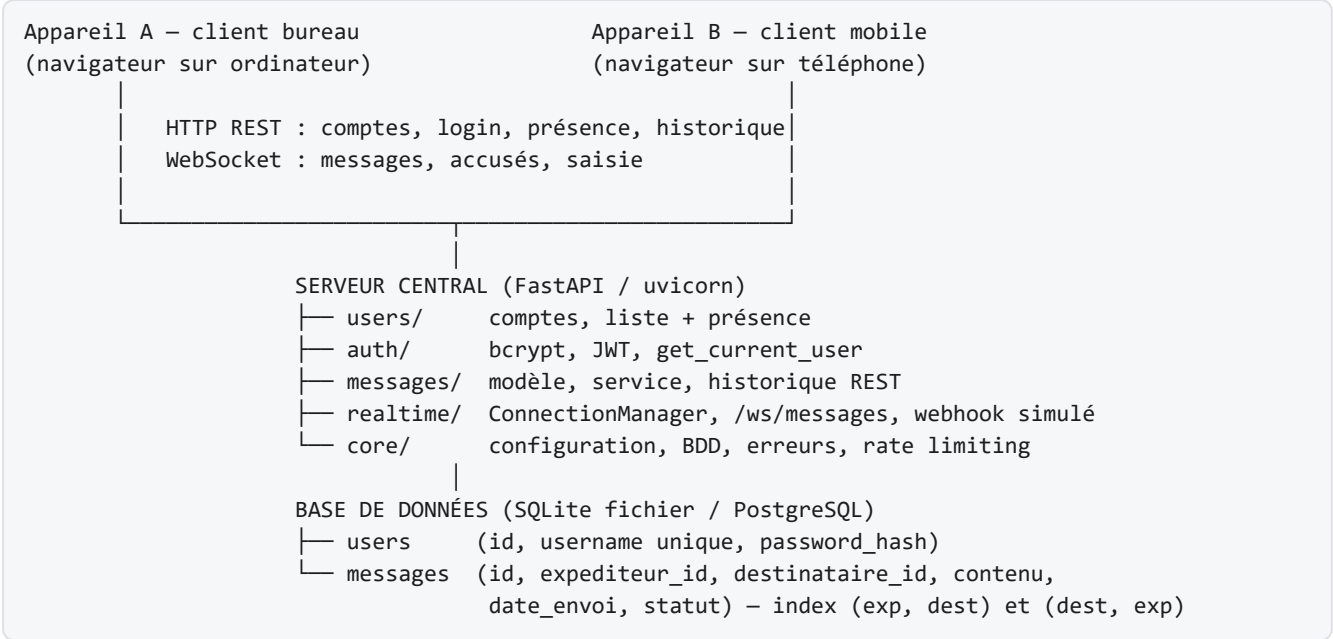
Fonctionnalités obligatoires couvertes :

Exigence de l'énoncé (§6)	Réalisation
Création de compte et authentification JWT	`POST /api/comptes`, `POST /api/auth/login` (bcrypt + JWT HS256)
Messages temps réel entre deux clients distincts	Canal `/ws/messages`, clients `desktop/` et `mobile/`
Persistance de tous les messages	Table `messages` (SQLite fichier, PostgreSQL possible)
Historique via l'API REST	`GET /api/messages/{utilisateurId}`
Protection des endpoints sensibles par JWT	Dépendance `get_current_user` → `401 Unauthorized`

Bonus réalisés : accusé de réception à trois états (`envoye` / `livre` / `lu`), indicateur de saisie, notification hors ligne par webhook simulé, limitation de débit, documentation OpenAPI/Swagger.

2. Architecture

2.1. Vue d'ensemble



Le serveur sert aussi les fichiers statiques des deux clients (`/clients/desktop/` , `/clients/mobile/`). Les clients sont donc chargés depuis la même origine que l'API, ce qui évite toute configuration d'adresse sur le téléphone : il suffit d'ouvrir `http://<IP du serveur>:8000/clients/mobile/` .

2.2. Chemin complet d'un message

1. Le client A envoie sur sa WebSocket : `{"type": "message", "destinataireId": 2, "contenu": "Bonjour !"}` .
2. Le serveur valide la trame avec un schéma Pydantic (contenu de 1 à 5000 caractères), vérifie la limite anti-spam et refuse un destinataire inexistant ou l'expéditeur lui-même.
3. L' `expediteurId` est **déduit du JWT** présenté à l'ouverture de la connexion, et la `dateEnvoi` est **générée par l'horloge du serveur** en UTC : un client ne peut ni usurper un autre utilisateur ni antidater un message.
4. Le message est **persisté** en base (statut `livre` si B est connecté, `envoye` sinon).
5. Il est **retransmis** immédiatement à B s'il est connecté : `{"type": "message", "id", "expediteurId", "destinataireId", "contenu", "dateEnvoi", "statut"}` — le format imposé par l'énoncé, complété de l' `id` et du `statut` .
6. A reçoit un accusé `{"type": "ack", ...}` qui confirme l'enregistrement côté serveur.
7. Quand B affiche la conversation, son client envoie `{"type": "read", "expediteurId": 1}` ; le serveur passe les messages à `lu` et prévient A (`read_ack` avec la liste des `messageIds`).

2.3. Répartition REST / WebSocket

Opération	Technologie	Endpoint
Création de compte, connexion	REST `POST`	`/api/comptes` , `/api/auth/login`
Liste des utilisateurs et présence	REST `GET`	`/api/utilisateurs`
Historique d'une conversation	REST `GET`	`/api/messages/{utilisateurId}`

Opération	Technologie	Endpoint
Envoi / réception en direct, accusés, saisie	WebSocket	`/ws/messages?token=<JWT>`

Pourquoi le WebSocket pour les messages, et pas le polling ? Avec du polling, le client B devrait interroger le serveur à intervalle régulier : soit l'intervalle est court et la quasi-totalité des requêtes reviennent vides (coût réseau et serveur), soit il est long et le message arrive avec retard. Le WebSocket est une connexion TCP persistante et bidirectionnelle, établie une seule fois par un handshake HTTP (`101 Switching Protocols`, vu au TP4), après lequel chaque trame ne coûte que quelques octets d'en-tête. Surtout, c'est **le serveur qui pousse** le message vers B dès qu'il le reçoit de A, sans requête préalable de B : c'est la seule manière d'obtenir une livraison immédiate.

La présence (`connecte`) est, conformément au tableau de l'énoncé, une consultation ponctuelle en REST : les clients la rafraîchissent toutes les 5 secondes et à chaque message reçu. C'est le seul endroit où nous acceptons une forme de polling, car un retard de quelques secondes sur un point vert est sans conséquence, contrairement à un retard sur un message.

2.4. Question : pourquoi l'historique est-il exposé en REST plutôt qu'en WebSocket ?

L'historique d'une conversation est une **ressource** au sens REST (Chapitre 4) : un ensemble de données stable, identifiable par une URL, `GET /api/messages/2` désignant « ma conversation avec l'utilisateur 2 ». Ce choix apporte les propriétés du modèle REST, qu'un canal WebSocket n'a pas :

- **Ressource identifiable par une URL.** L'historique peut être demandé par n'importe quel outil HTTP (navigateur, curl, Bruno, Postman, un futur client natif) sans connaître le protocole de trames du WebSocket. Il est documenté automatiquement dans Swagger, alors qu'un échange WebSocket n'a pas d'équivalent standard dans OpenAPI.
- **Méthode sûre et idempotente.** `GET` ne modifie rien : on peut le rejouer sans risque après une coupure réseau, ce que fait justement notre client à chaque reconnexion pour rattraper ce qu'il a manqué. Sur un WebSocket, il faudrait inventer un protocole de « demande d'historique » et de corrélation des réponses.
- **Cache.** Une réponse `GET` peut être mise en cache par le navigateur ou un proxy (`Cache-Control`, `ETag` / `304 Not Modified`), alors que les trames WebSocket ne passent par aucun mécanisme de cache. Pour un historique qui ne change qu'en fin de liste, un cache validé par `ETag` éviterait de retransférer toute la conversation. Nous ne l'avons pas activé, car la réponse contient le statut de lecture des messages, qui évolue ; mais l'architecture le permet.
- **Sans état.** Chaque requête porte son JWT et se suffit à elle-même : n'importe quelle instance du serveur peut y répondre, alors que la connexion WebSocket est attachée à un processus précis (voir §6.1).
- **Évolutivité.** Une pagination (`?avant=<id>&limite=50`) s'ajouterait naturellement en paramètres d'URL, comme au TP1 (`_page`, `_limit`) et au TP2 (`Pageable`). Elle n'est pas implémentée, nos conversations de démonstration étant courtes.

À l'inverse, le WebSocket est un **flux d'événements** : il ne transporte que ce qui se passe pendant que la connexion est ouverte. S'il était la seule source de l'historique, un client déconnecté perdrait tout ce qui a été dit en son absence — exactement le constat fait au TP4 avec le service d'écho de Postman, où l'historique disparaît à la reconnexion.

3. Sécurité

3.1. Mots de passe

Les mots de passe sont hachés avec **bcrypt** (`pwdlib[bcrypt]`), qui intègre un sel aléatoire par utilisateur et un coût de calcul volontairement élevé : deux comptes ayant le même mot de passe ont des empreintes différentes, et une attaque par dictionnaire hors ligne devient lente. Le mot de passe n'apparaît jamais dans une réponse (le schéma de sortie ne contient que `id` et `username`) ni dans les logs. Sa longueur est validée (4 à 72 caractères : bcrypt ignore tout ce qui dépasse 72 octets).

3.2. Cycle de vie du JWT

1. **Création** au login : jeton HS256 signé avec un secret lu dans `.env` (jamais dans le code), contenant `sub` (id utilisateur, sous forme de chaîne conformément à la RFC 7519), `iat` et `exp` (60 minutes).
2. **REST** : en-tête `Authorization: Bearer <jeton>`. La dépendance `get_current_user` renvoie `401 Unauthorized` avec `WWW-Authenticate: Bearer` si le jeton est absent, mal formé, falsifié, expiré, ou si l'utilisateur n'existe plus. Nous avons utilisé `HTTPBearer(auto_error=False)` : il déclare le schéma dans OpenAPI (bouton « Authorize » de Swagger), mais nous levons nous-mêmes l'erreur, car selon les versions FastAPI renvoie `403` alors que l'énoncé exige `401`.
3. **WebSocket** : l'API WebSocket des navigateurs ne permet pas d'ajouter un en-tête `Authorization` au handshake ; le jeton est donc passé en paramètre de requête (`/ws/messages?token=...`). Il est vérifié **avant** d'enregistrer la connexion. En cas d'échec, le serveur accepte puis ferme immédiatement la connexion avec le code **1008 (Policy Violation, RFC 6455 §7.4.1)** : fermer avant d'accepter produit un simple refus HTTP 403 du handshake, que le navigateur ne voit que comme un code 1006 sans motif. Avec 1008, le client sait que sa session a expiré et renvoie l'utilisateur à l'écran de connexion.

Limite connue : un jeton en paramètre d'URL apparaît dans les logs d'accès du serveur. En production, on utiliserait un jeton à usage unique et de très courte durée, obtenu en REST juste avant d'ouvrir le WebSocket, ou un cookie `HttpOnly`.

Nous n'avons pas implémenté de *refresh token* (pratiqué au TP2) : au bout d'une heure, l'utilisateur se reconnecte simplement.

3.3. Autres protections

- **Limitation de débit (TP5)** : fenêtre glissante en mémoire (`core/rate_limit.py`), même algorithme que le `RateLimitFilter` du TP5. Sur le WebSocket, au plus 10 messages par 10 secondes par utilisateur : au-delà, le message n'est ni persisté ni transmis, et le client reçoit `{"type": "error", "code": "RATE_LIMITED", "retryAfter": n}`. Sur le login, 5 tentatives par minute par couple (IP, nom d'utilisateur), puis `429 Too Many Requests` avec l'en-tête `Retry-After` : c'est une protection contre le brute-force des mots de passe.
- **Validation des entrées** : schémas Pydantic sur tout le JSON reçu, en REST comme en WebSocket. Une trame qui n'est pas du JSON reçoit une erreur sans couper la connexion.
- **Injection** : SQLAlchemy génère des requêtes paramétrées (pas d'injection SQL). Côté client, les messages sont insérés avec `textContent`, jamais `innerHTML` : un message contenant du HTML ou un `<script>` s'affiche tel quel (pas de XSS, cf. TP5).
- **Format d'erreur homogène (TP2, TP5)** : toutes les erreurs REST ont la forme `{"detail", "code"}` (`401 UNAUTHORIZED`, `404 NOT_FOUND`, `409 CONFLICT`, `422 VALIDATION_ERROR` avec le détail par champ, `429 RATE_LIMITED`), et les erreurs WebSocket la forme `{"type": "error", "code", "detail"}`.

- **HTTPS** : non mis en place pour la démonstration, qui se déroule sur un réseau local en HTTP/WS. En production, on placerait un reverse proxy (Nginx, Caddy) terminant TLS devant uvicorn, comme le certificat `keystore.p12` du TP5 : HTTPS pour le REST et **WSS** pour le WebSocket. C'est indispensable ici, puisque le mot de passe et le JWT circulent dans les requêtes.

4. Question : que faire si le destinataire n'est pas connecté ?

4.1. Comportement retenu

Le message **ne doit pas être perdu**. Notre serveur applique la règle « persister d'abord, livrer ensuite » :

1. **Persistance systématique.** Le message est enregistré en base avant toute tentative de livraison, avec le statut `envoye`. L'expéditeur reçoit un `ack` avec ce statut : son client affiche « Envoyé » sous la bulle, ce qui signifie « stocké sur le serveur, pas encore remis ».
2. **Notification hors ligne (bonus).** Le serveur déclenche un webhook vers un service tiers de notifications, ici simulé par un log : `Webhook simulé : notification envoyée à l'utilisateur 2 (nouveau message 3 de l'utilisateur 1)`. C'est le modèle du TP4 (webhook.site) : le serveur prévient un système externe qu'un événement a eu lieu ; dans une vraie application, ce service enverrait une notification push au téléphone.
3. **Livraison à la reconnexion.** Quand B ouvre sa connexion WebSocket, le serveur passe ses messages en attente à `livre` et envoie à chaque expéditeur connecté un `{"type": "delivered_ack", "messageIds": [...]}` : le statut devient « Distribué » chez A.
4. **Rattrapage par l'historique.** Le client de B recharge la conversation en REST (`GET /api/messages/1`) ; il obtient tous les messages, y compris ceux envoyés en son absence, dans l'ordre chronologique. Dès qu'il les affiche, il envoie `read` et A voit « Lu ».

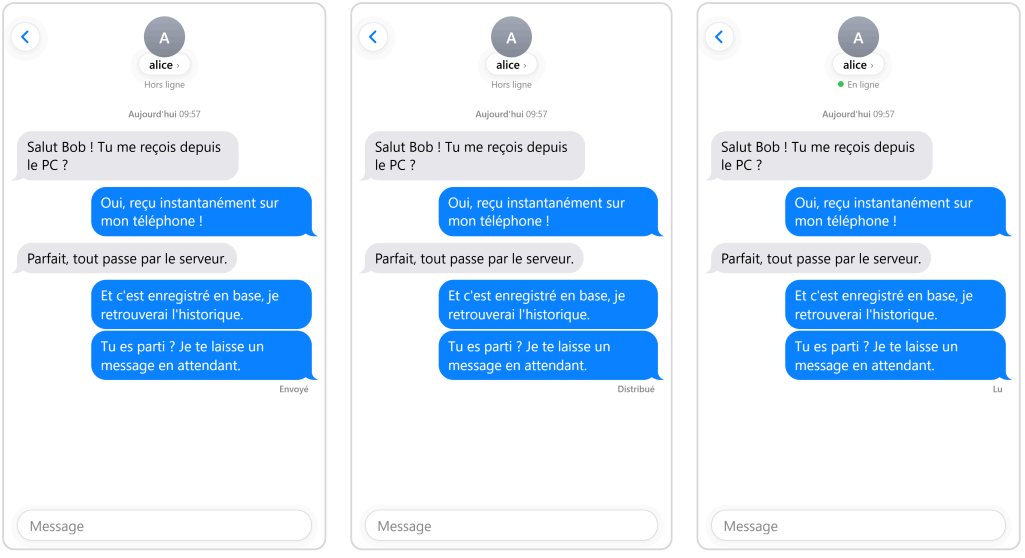


Figure 1 — Côté expéditeur (bob, mobile) : alice est hors ligne, le message reste « Envoyé » (stocké sur le serveur) ; à la reconnexion d'alice il passe à « Distribué » (`delivered_ack`), puis à « Lu » quand elle rouvre la conversation.

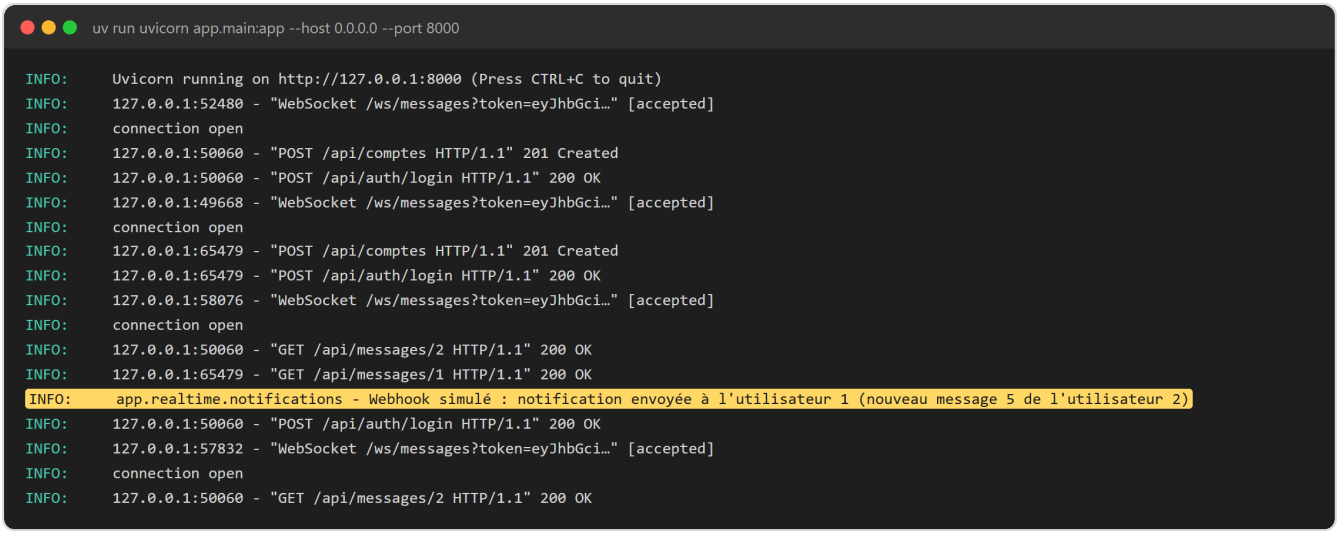


Figure 2 — Console du serveur pendant le même scénario : création des comptes et login (REST), ouverture des WebSockets, chargement de l'historique, et webhook simulé déclenché par le message envoyé à alice hors ligne. Les jetons JWT présents dans l'URL du WebSocket ont été tronqués (§3.2).

4.2. Parallèle avec Apache Kafka (Chapitre 6)

Caractéristique	WebSocket seul	Broker classique (AMQP)	Apache Kafka	Notre architecture
Persistance	Aucune (mémoire)	File vidée à l'acquittement	Journal append-only sur disque	Table SQL `messages` sur disque
Destinataire hors ligne	**Message perdu**	Conservé dans la file (TTL)	Conservé selon la rétention	Conservé jusqu'à suppression
Position de lecture	Aucune	Gérée par le broker	`offset` géré par le consommateur	Statut par message + historique complet
Relecture	Impossible	Impossible	Possible (reset de l'offset)	Possible via `GET`

Dans Kafka, la déconnexion d'un consommateur n'empêche pas le producteur de publier : les événements sont écrits durablement dans une partition, et le consommateur reprend à son `offset` quand il revient. Notre table `messages` joue le même rôle de **journal durable** : la livraison par WebSocket est une optimisation de latence, et la base de données reste la source de vérité. Le statut `envoye` / `livre` joue le rôle d'un offset par destinataire : il indique ce qui reste à remettre.

Il reste une différence : Kafka garantit la remise aux consommateurs, alors que notre envoi WebSocket est « au mieux ». Si la connexion de B meurt entre la persistance et l'envoi, le message est marqué `livre` sans avoir été affiché. Il n'est pas perdu pour autant, puisque B le retrouvera dans l'historique au prochain chargement. Une garantie plus forte nécessiterait un accusé de réception explicite du client destinataire avant de passer à `livre`.

5. Choix techniques

Sujet	Choix	Justification
Langage, framework	Python 3.13, **FastAPI** + uvicorn	Accord de l'enseignant pour ne pas utiliser Spring Boot. REST et WebSocket natifs dans le même serveur, validation JSON par Pydantic, OpenAPI généré automatiquement.

Sujet	Choix	Justification
Dépendances	`uv` (`pyproject.toml` + `uv.lock`)	Environnement reproductible pour le binôme et le correcteur.
ORM	SQLAlchemy 2.0 (synchrone)	Standard Python, requêtes paramétrées, changement de SGBD par simple `DATABASE_URL`.
Base de données	**SQLite fichier**	Équivalent Python de H2 accepté par l'énoncé. Un *fichier* et non une base en mémoire, pour que l'historique survive à un redémarrage du serveur.
Format	JSON en **camelCase** côté API, snake_case côté Python	L'énoncé impose `expediteurId`, `dateEnvoi`, `motDePasse` : `alias_generator=to_camel` dans les schémas Pydantic.
Dates	UTC, ISO 8601, générées par le serveur	Les horloges des appareils ne sont pas fiables ; un `TypeDecorator` réattache le fuseau UTC que SQLite perd.
Clients	HTML + JavaScript sans framework	Le front n'est pas évalué sur le design. `common/api.js` (appels REST et WebSocket) et `common/chat.js` (logique d'interface) sont partagés ; `desktop/` (contacts et conversation côte à côte) et `mobile/` (un écran à la fois, boutons larges) n'ont en propre que leur mise en page. L'interface reprend les codes visuels de Messages (iMessage) sur macOS et iOS, définis une fois dans `common/imessage.css` (couleurs système, mode sombre, bulles groupées), sans fonctionnalité factice.
Tests	pytest + `TestClient` (REST et WebSocket), collection Bruno	Automatiser ce que nous aurions testé à la main ; Bruno reprend l'outil des TP1 et TP2.

5.1. Réutilisation des notions des TP

TP	Notion pratiquée	Utilisation dans le projet
TP1 — HTTP et Postman	Méthodes, codes de statut, en-têtes, Bearer, tests automatisés	`201`, `401`, `404`, `409`, `422`, `429` ; `Authorization: Bearer` ; collection Bruno avec assertions et tests (statut, champs, temps de réponse < 1 s)
TP2 — API REST Spring Boot	Validation, format d'erreur unifié, BCrypt, JWT, rate limiting, Swagger	Validation Pydantic, `{detail, code}`, bcrypt, JWT, limitation de débit, Swagger avec « Authorize »
TP3 — Formats et REST	JSON, URLs orientées ressources, idempotence	JSON exclusivement, `/api/messages/{id}`, `POST` non idempotent (un second `POST /api/comptes` donne `409`)
TP4 — Temps réel	WebSocket (`101`), perte de l'historique à la reconnexion, webhooks, Kafka	Canal `/ws/messages`, historique REST + BDD, webhook simulé hors ligne, parallèle Kafka (§4.2)
TP5 — Sécurité	JWT, 401, fenêtre glissante, `429` + `Retry-After`, XSS, HTTPS	`get_current_user`, `core/rate_limit.py`, affichage par `textContent`, HTTPS discuté en §3.3

L'énoncé fait référence au « TP4 » pour le JWT et le rate limiting. Dans notre progression, ces notions ont été pratiquées au TP2 (exercices 2 et 3) et au TP5 ; le TP4 portait sur le temps réel.

6. Difficultés rencontrées

6.1. État en mémoire et limitation à un worker

Le `ConnectionManager` garde les connexions actives dans un dictionnaire Python en mémoire du processus (`dict[int, WebSocket]`), de même que les compteurs anti-spam. Avec plusieurs workers uvicorn ou plusieurs serveurs derrière un répartiteur de charge, chaque processus a sa propre mémoire : si A est connecté au worker 1 et B au worker 2, le worker 1 considère B comme hors ligne. Nous lançons donc un seul worker, ce qui suffit largement pour la démonstration. Pour passer à l'échelle, il faudrait un bus partagé (**Redis Pub/Sub**) : chaque worker s'abonne aux canaux des utilisateurs qu'il héberge, et un message est publié sur le bus pour être repris par le worker qui détient la connexion du destinataire. Les compteurs de rate limiting iraient eux aussi dans Redis.

6.2. ORM synchrone dans un serveur asynchrone

FastAPI repose sur `asyncio` : une boucle d'événements sur un seul thread sert toutes les connexions WebSocket. Un `db.commit()` synchrone appelé directement dans le handler `async def` bloquerait cette boucle, donc tous les utilisateurs, pendant l'écriture sur disque. Tous les accès à la base depuis le WebSocket passent par `run_in_threadpool`, qui exécute le code bloquant dans un pool de threads. Les routes REST, déclarées en `def`, sont déjà exécutées dans ce pool par FastAPI.

6.3. Bugs découverts lors de la validation

En rejouant le scénario de démonstration contre un vrai serveur (et non plus seulement avec `TestClient`), nous avons trouvé quatre défauts. Chacun est désormais couvert par un test :

- **Reconnexion rapide.** Quand un téléphone perd le réseau puis se reconnecte avant que le serveur ait détecté la mort de l'ancienne socket, la fermeture tardive de l'ancienne retirait du registre… la nouvelle connexion. L'utilisateur apparaissait hors ligne et ne recevait plus rien. `disconnect(user_id, ws)` ne retire désormais l'entrée que si c'est bien la même socket.
- **Destinataire inexistant.** SQLite ne vérifie pas les clés étrangères par défaut : un message vers l'utilisateur 42, qui n'existe pas, était accepté et stocké. Le serveur vérifie maintenant le destinataire et répond `DESTINATAIRE_INCONNU`.
- **Trame invalide.** Une trame qui n'était pas du JSON faisait lever une exception à `receive_json()` et coupait la connexion. Nous lisons maintenant le texte et le décodons nous-mêmes, pour renvoyer une erreur `JSON_INVALIDE`.
- **Code 1008 invisible.** Le refus d'authentification était émis avant `accept()`. Dans les tests il apparaissait bien comme 1008, mais un vrai navigateur ne voyait qu'un échec de handshake (voir §3.2).

6.4. Déconnexions silencieuses

Sur mobile, une perte de réseau ou une mise en veille ne produit pas toujours de trame de fermeture. Toute écriture qui échoue sur une socket retire l'utilisateur du registre, ce qui maintient la présence exacte. Côté client, une fermeture inattendue déclenche une reconnexion automatique au bout de 3 secondes, suivie du rechargement de l'historique pour récupérer les messages manqués.

7. Validation et démonstration

7.1. Tests automatisés

- **pytest, 43 tests** (`backend/tests/`) :
- hachage et JWT (valide, falsifié, expiré) ;
- `401` sur chaque endpoint protégé, `409`, `422` au format homogène, `429` avec `Retry-After` ;
- historique (ordre chronologique, `404`) ;
- registre de présence, y compris la reconnexion rapide ;
- WebSocket : refus `1008`, livraison directe, accusés `livre` / `lu`, `delivered_ack` à la reconnexion, destinataire hors ligne puis historique, webhook simulé, anti-spam, trames invalides.
- **Collection Bruno** (`docs/bruno/`) : 11 requêtes REST, 12 tests et 15 assertions, toutes au vert.

7.2. Scénario de démonstration (§2.1 de l'énoncé)

Le scénario a été rejoué de bout en bout sur **deux PC distincts** connectés au même réseau Wi-Fi : le premier exécute le serveur (`uvicorn --host 0.0.0.0`) et le client bureau, le second ouvre le client mobile depuis son navigateur à l'adresse IP du premier.

Étape	Résultat observé	Captures
1–2. Connexion de A (bureau) puis de B (mobile)	Chacun voit l'autre « En ligne » (point vert)	Fig. 4, 5
3. A envoie un message	Il apparaît instantanément sur B (point bleu « non lu ») ; « Distribué », puis « Lu » quand B ouvre la conversation	Fig. 6
4. B répond	Bulle « … » chez A pendant la saisie, puis le message apparaît instantanément sur A	Fig. 7, 3
5. A se déconnecte et se reconnecte	B le voit « Hors ligne » ; un message de B affiche « Envoyé » et le webhook est loggé ; à la reconnexion, l'historique complet est rechargé et le message passe à « Lu »	Fig. 1, 2, 8

7.3. Capture de la démonstration

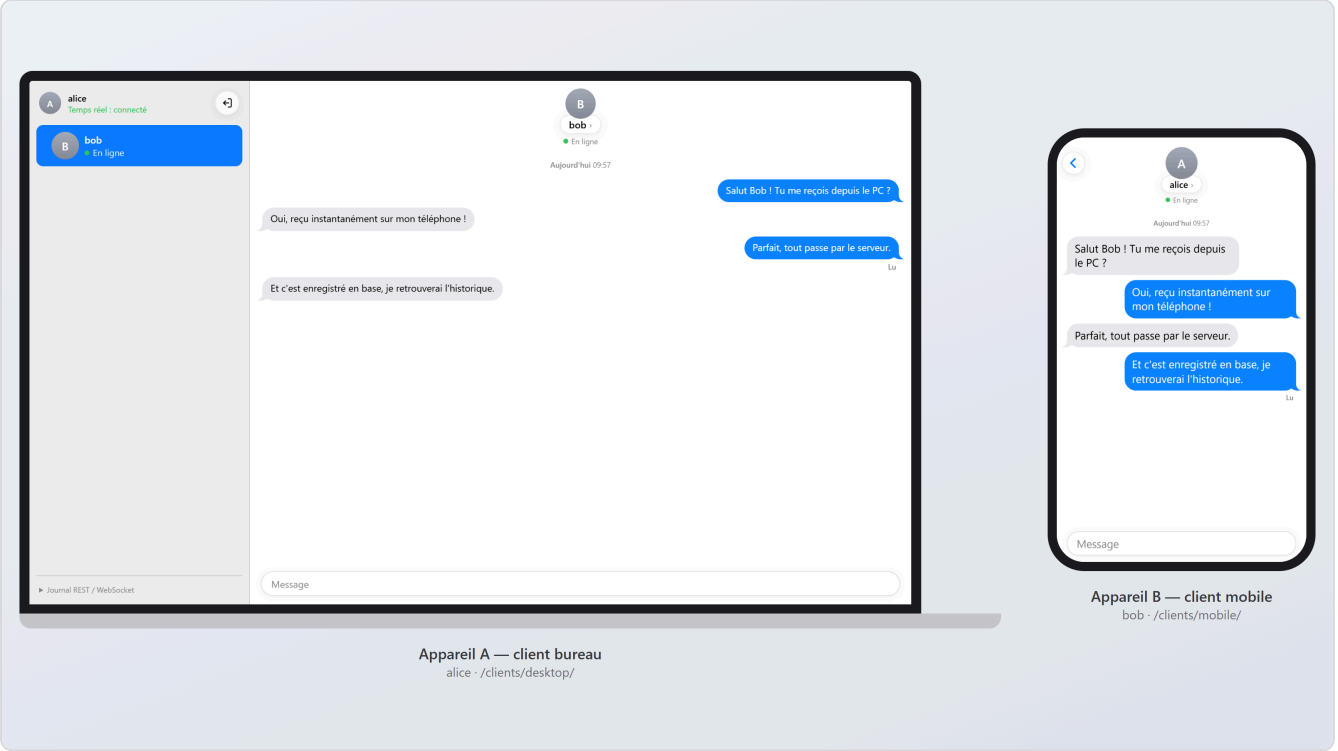


Figure 3 — Fin de l'étape 4 : la même conversation sur le client bureau (alice, à gauche) et sur le client mobile (bob, à droite). Chaque message envoyé par l'un apparaît sur l'autre, et le dernier message envoyé de chaque côté est marqué « Lu ».

Les captures de ce rapport ont été produites en rejouant automatiquement le scénario (Playwright) contre le serveur réel, avec deux sessions de navigateur indépendantes : une fenêtre de bureau (1280 × 760) pour A et un gabarit de téléphone (iPhone 13) pour B. Les autres captures sont regroupées en annexe.

8. Limites et pistes d'amélioration

- HTTPS/WSS à mettre en place derrière un reverse proxy (§3.3).
- Passage à plusieurs instances avec Redis Pub/Sub (§6.1).
- Pagination de l'historique et cache HTTP validé par `ETag` (§2.4).
- Plusieurs connexions simultanées pour un même compte (aujourd'hui, la plus récente remplace la précédente).
- Présence poussée en WebSocket plutôt que rafraîchie en REST, si l'on veut un indicateur instantané.
- Refresh token pour éviter la reconnexion au bout d'une heure.

Annexe — Captures du scénario

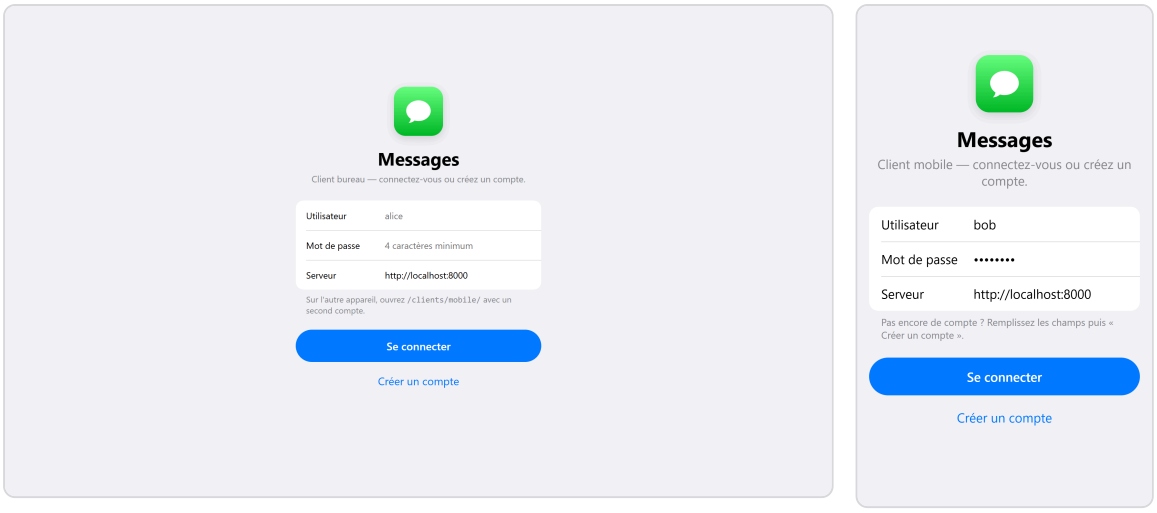


Figure 4 — Écrans de connexion. Le même formulaire sert à se connecter (`POST /api/auth/login`) et à créer un compte (`POST /api/comptes`, puis login). Le champ « Serveur » est prérempli avec l'origine de la page, servie par FastAPI.

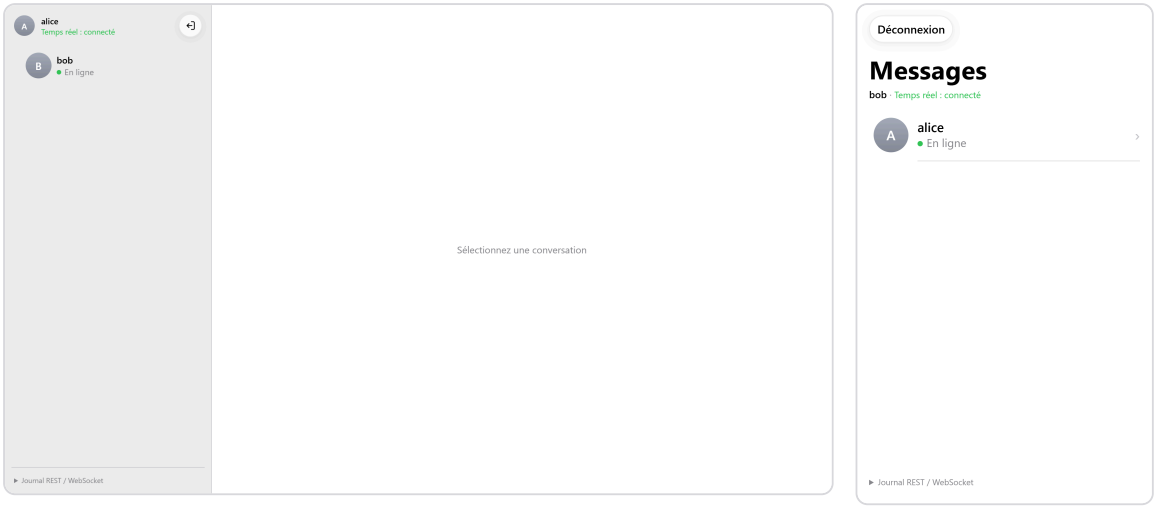


Figure 5 — Présence (`GET /api/utilisateurs`, champ `connecte`) : chacun voit l'autre « En ligne », avec le point vert.

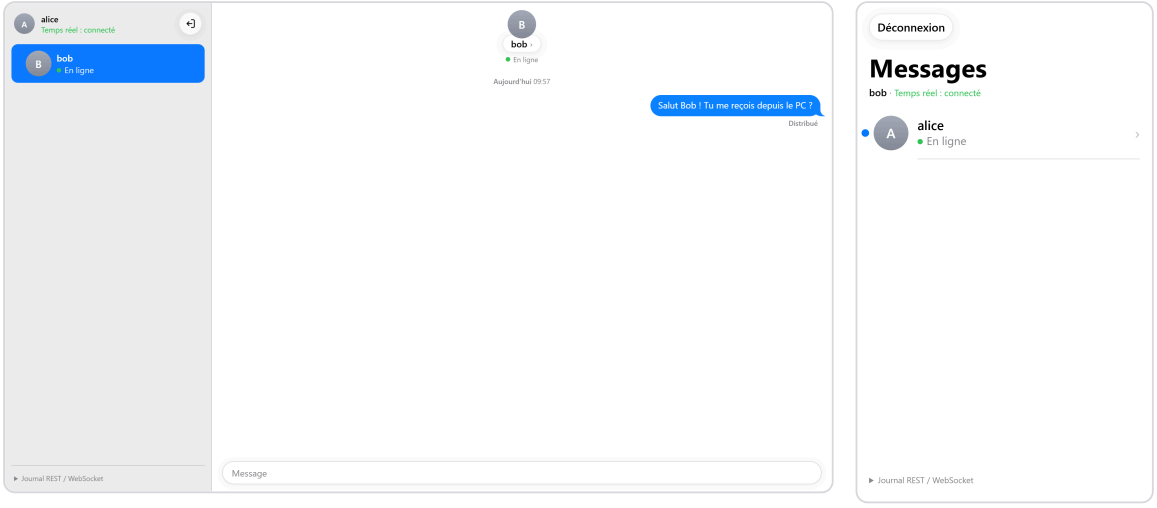


Figure 6 — Étape 3 : le message d'alice est poussé par le WebSocket vers bob, qui ne l'a pas encore ouvert (point bleu « non lu »). Chez alice, le statut est « Distribué ».

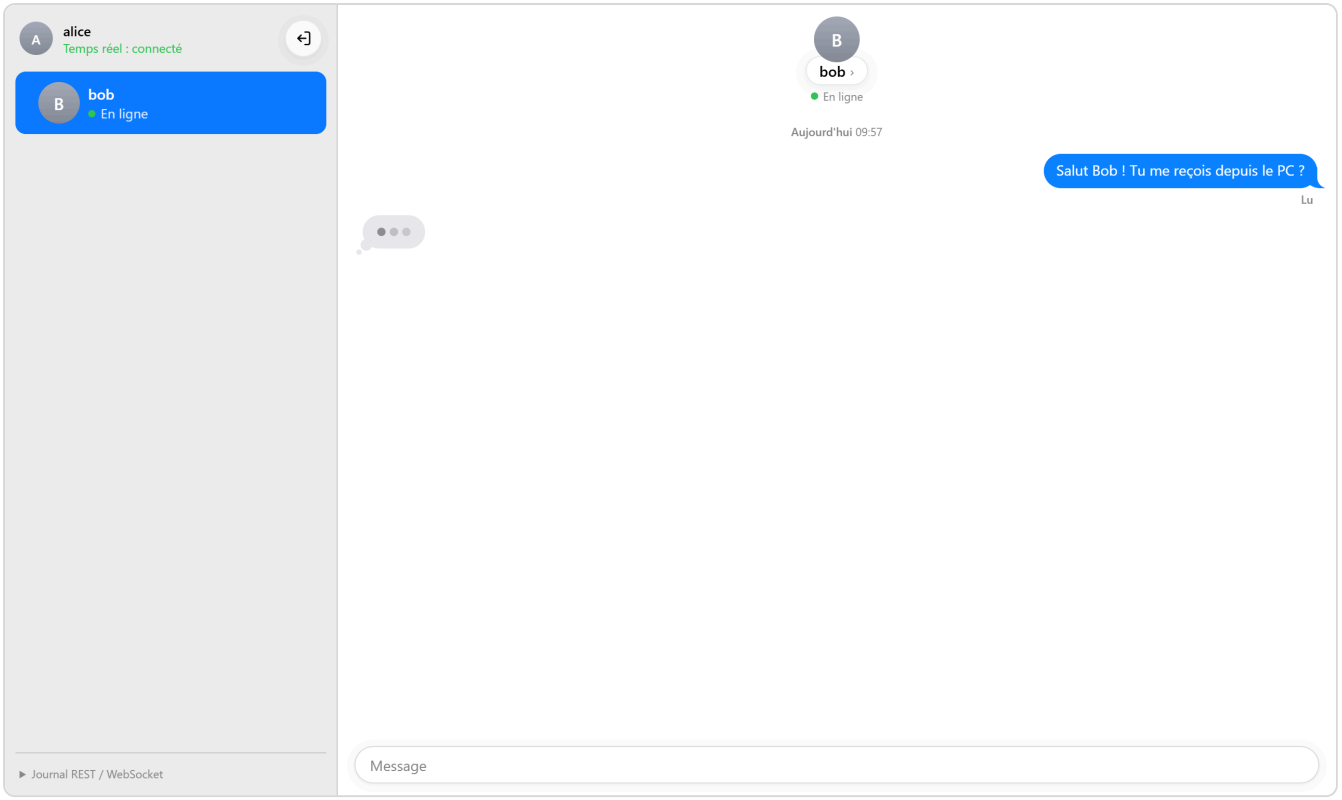


Figure 7 — Indicateur de saisie (bonus) : pendant que bob tape sa réponse, alice voit la bulle « … » (trames `typing`). Son premier message est passé à « Lu » quand bob a ouvert la conversation.

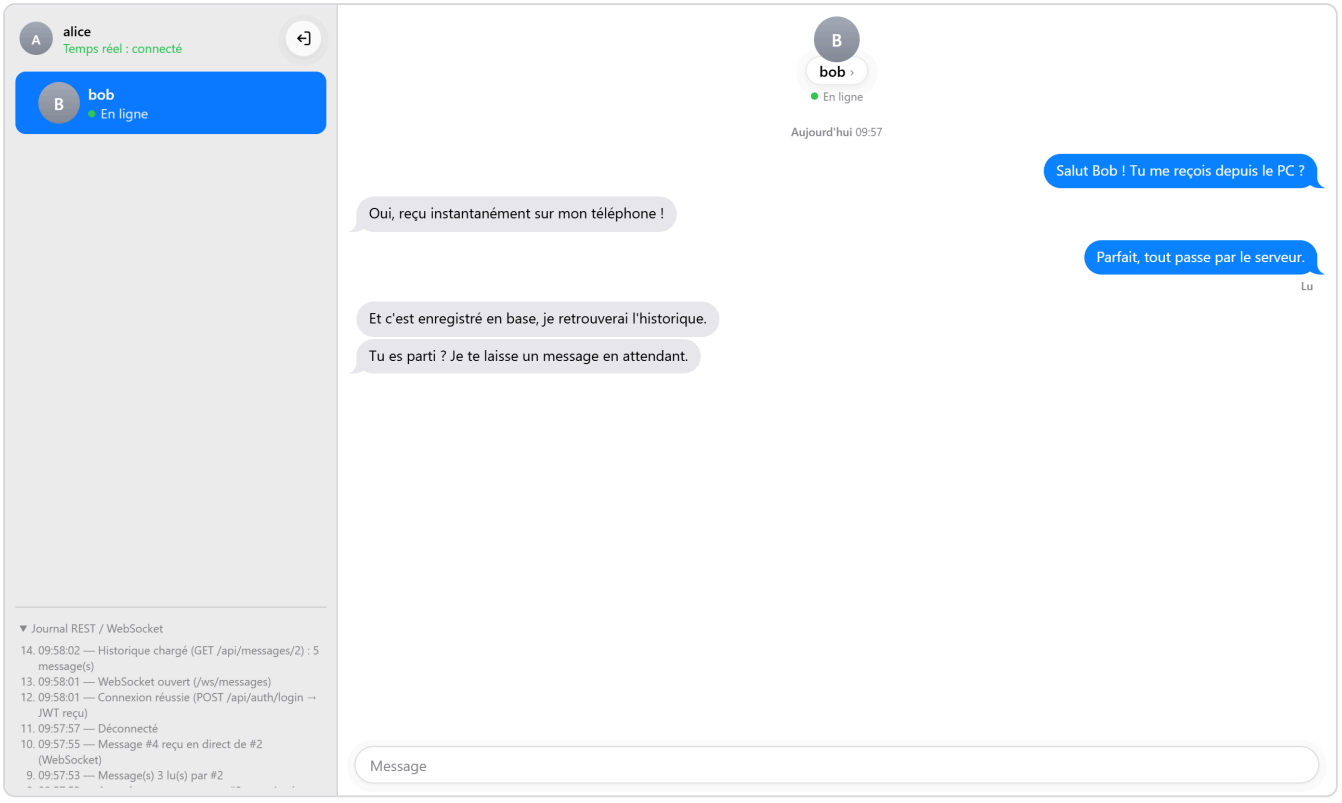


Figure 8 — Étape 5 : après déconnexion puis reconnexion d'alice, l'historique complet (5 messages, dont celui reçu hors ligne) est rechargé par `GET /api/messages/2`. Le journal déplié en bas à gauche montre la séquence : login REST, ouverture du WebSocket, historique REST.

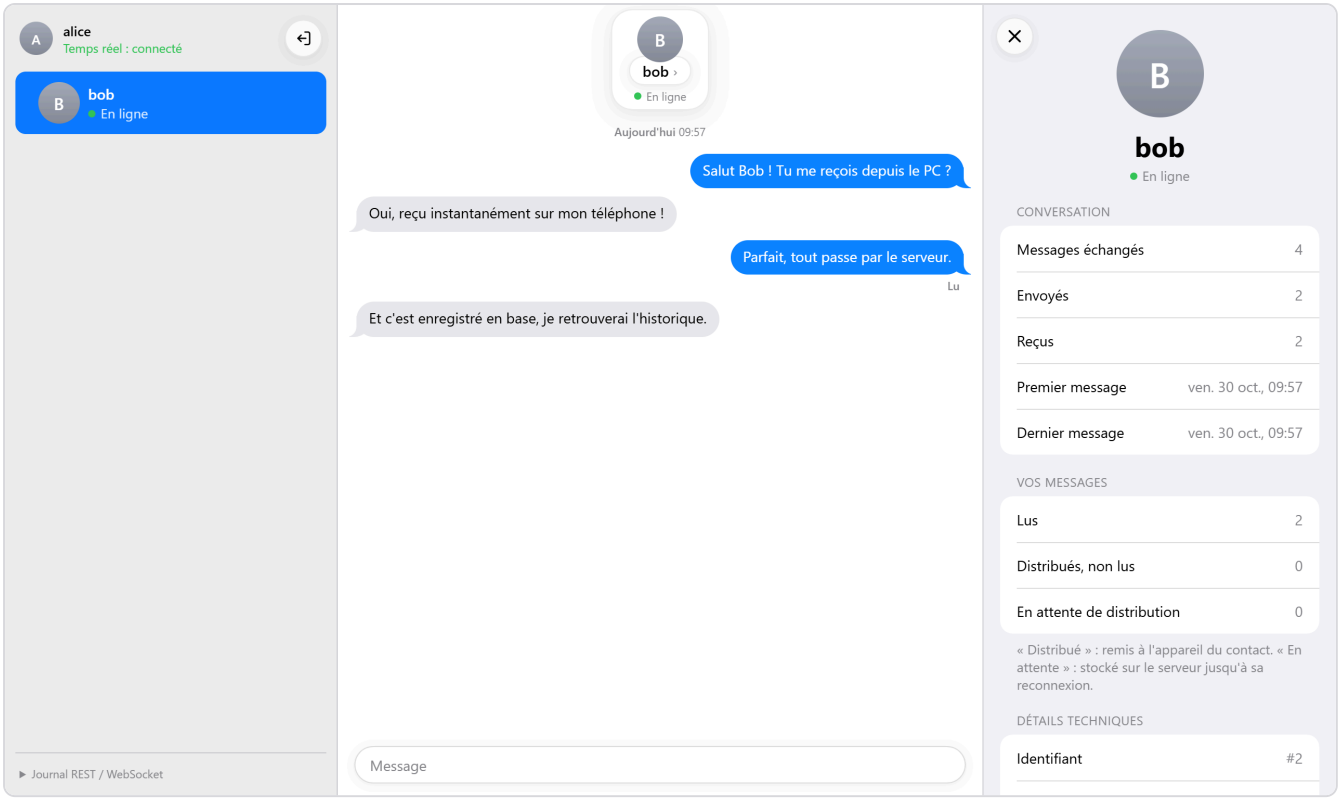


Figure 9 — Panneau d'informations sur le contact : statistiques calculées à partir de l'historique réel (envoyés, reçus, lus, distribués, en attente) et état du canal temps réel.

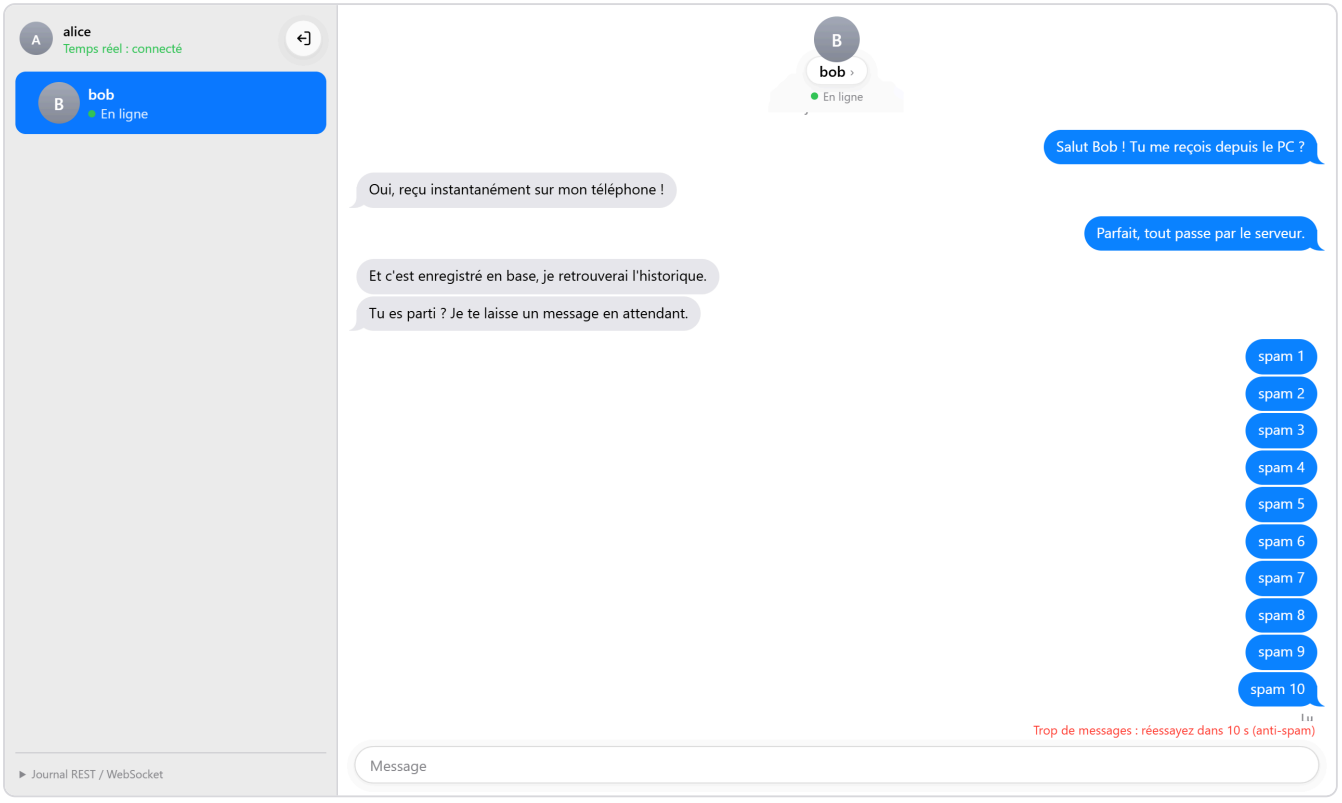


Figure 10 — Limitation de débit (§3.3) : sur 11 messages envoyés en rafale, les 10 premiers sont acceptés ; le 11ᵉ n'est ni persisté ni transmis, et le serveur répond `RATE_LIMITED` avec le délai d'attente.

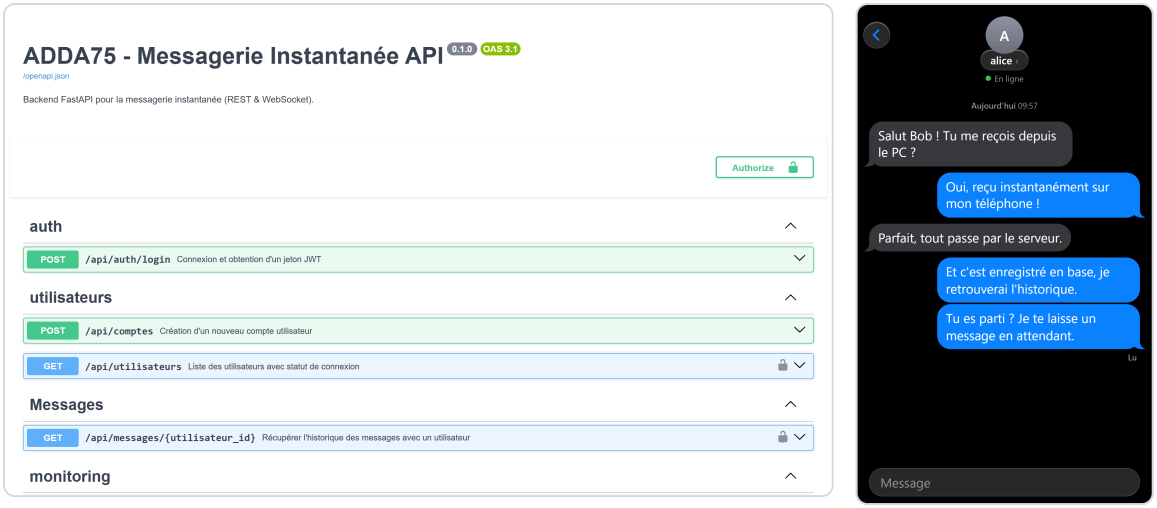


Figure 11 — À gauche, la documentation OpenAPI générée par FastAPI (`/docs`), avec le bouton « Authorize » pour le jeton Bearer. À droite, le client mobile en mode sombre.